CYNLR – Software Engineer Project Overview :

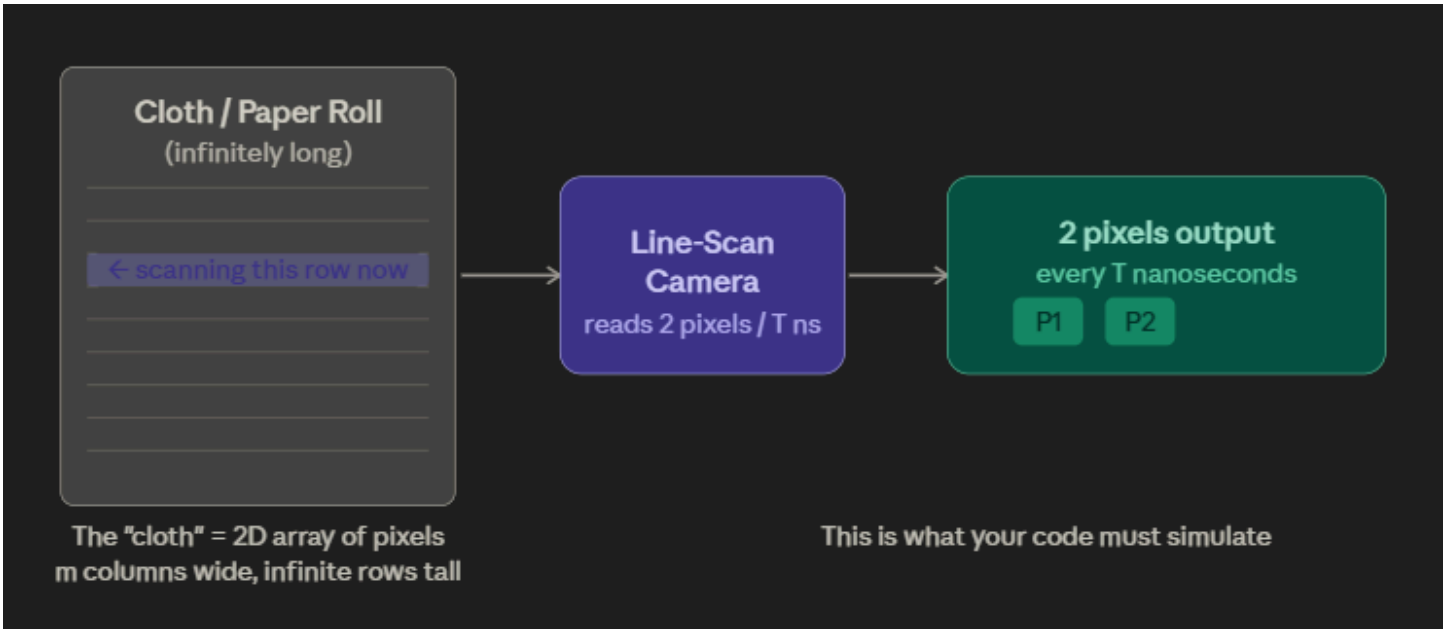


Image 1 in plain English: Think of a long cloth roll on a factory machine. A camera scans it line by line (row by row). For each scan, it reads **2 pixel values at a time**, every T nanoseconds. Your code must simulate this — Block 1 pretends to be that camera, and Block 2 checks each pixel for defects. The two blocks run **in parallel** (simultaneously), like two workers on an assembly line.

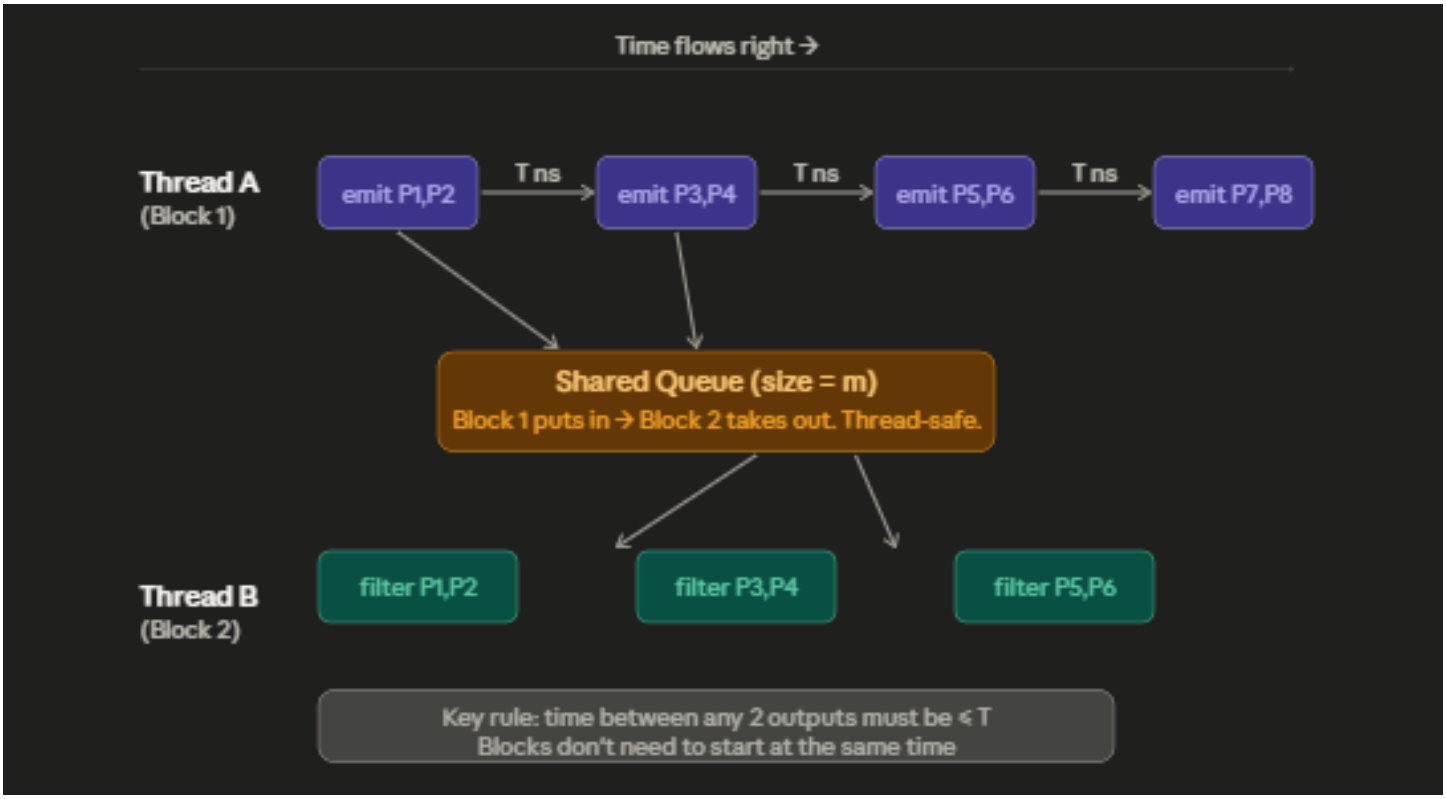


Image 2 in plain English: This shows that Block 1 and Block 2 run **at the same time** (parallel threads). Block 1 is like a chef cooking dishes, and Block 2 is a waiter serving them. They don't wait for each other to fully finish — while Block 2 is serving dish 1, Block 1 is already cooking dish 2. The **queue** in the middle is the counter where dishes are placed and picked up. The rule is: each dish must be ready within T nanoseconds.

Data Generation Block: How Block 1 works :

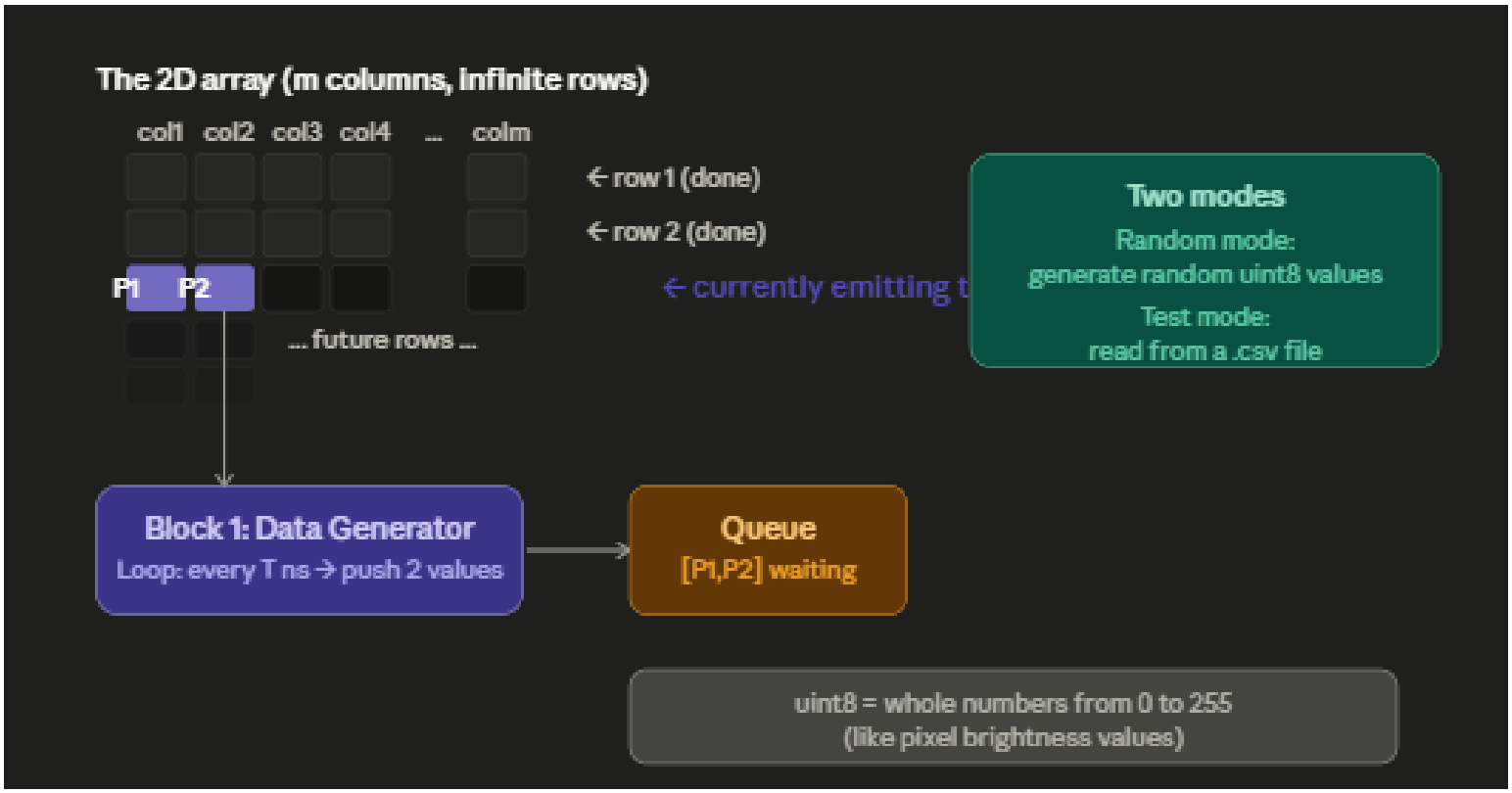


Image 3 in plain English: Block 1 reads the 2D array (like a spreadsheet) going left to right, top to bottom — but **only 2 cells at a time**. Every T nanoseconds it picks up the next 2 cells and pushes them into the queue. It's like reading a book two letters at a time on a strict timer. In **test mode** it reads from your CSV file. In **random mode** it just generates two random numbers each time.

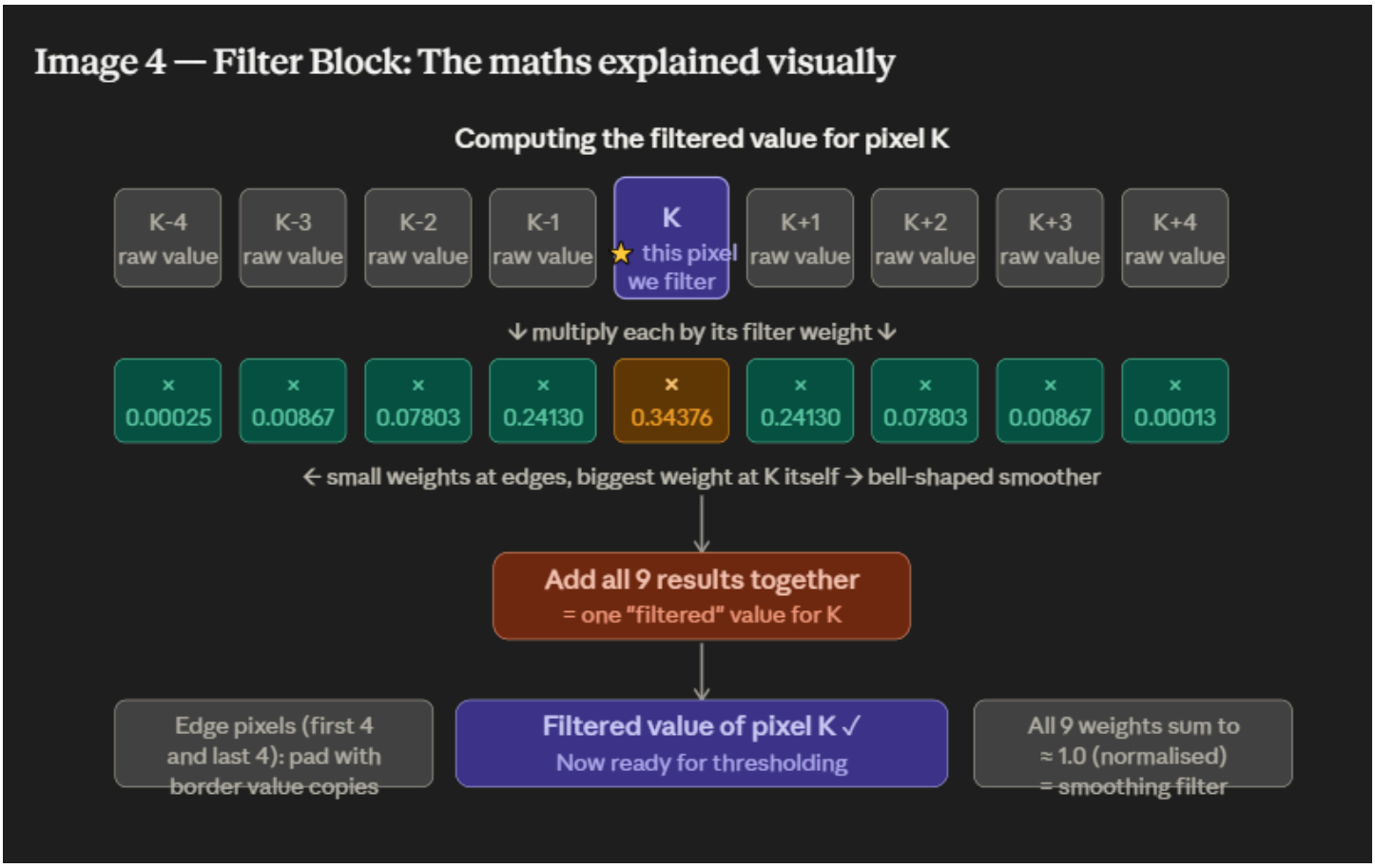


Image 4 in plain English: For every pixel K you want to filter, you look at its 4 neighbours to the left and 4 to the right — a window of 9 pixels total. You multiply each neighbour by a special weight (bigger weight for closer neighbours, tiny weight for far ones — a bell shape). Then add all 9 results. This is called **convolution** and it's basically a **smoothing operation** — it removes noise (random spikes) from the signal, like blurring a jagged line into a smooth curve.

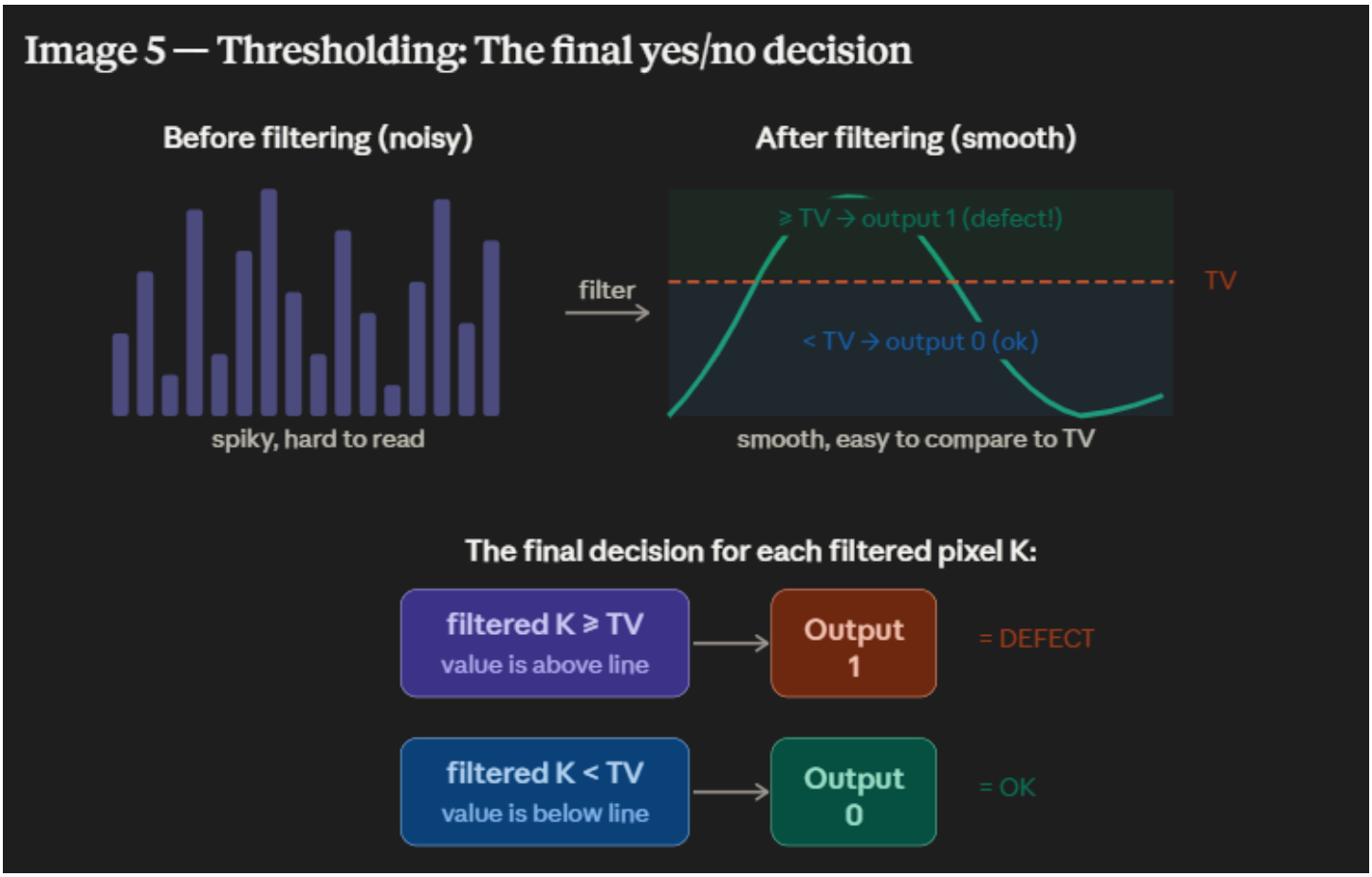
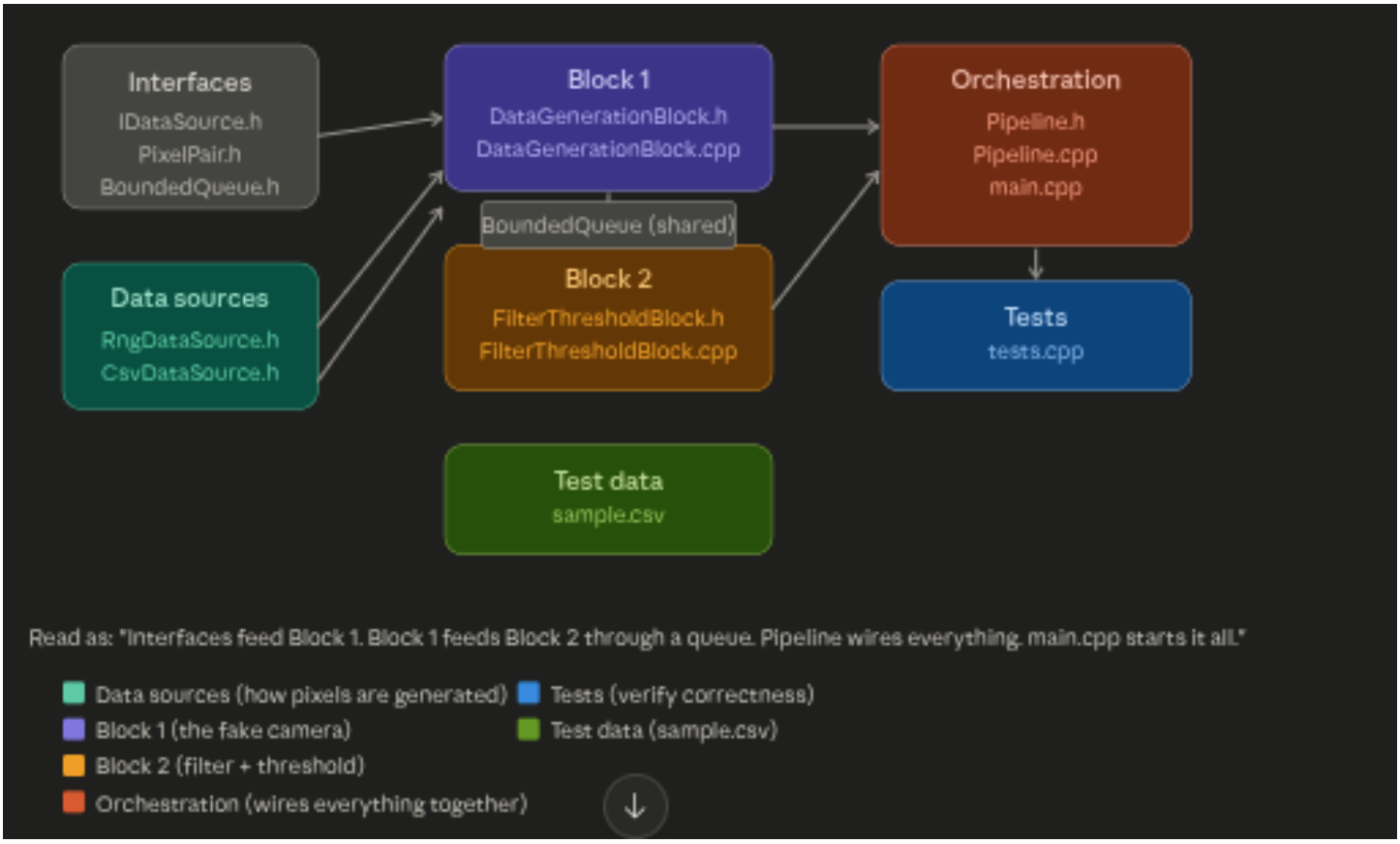
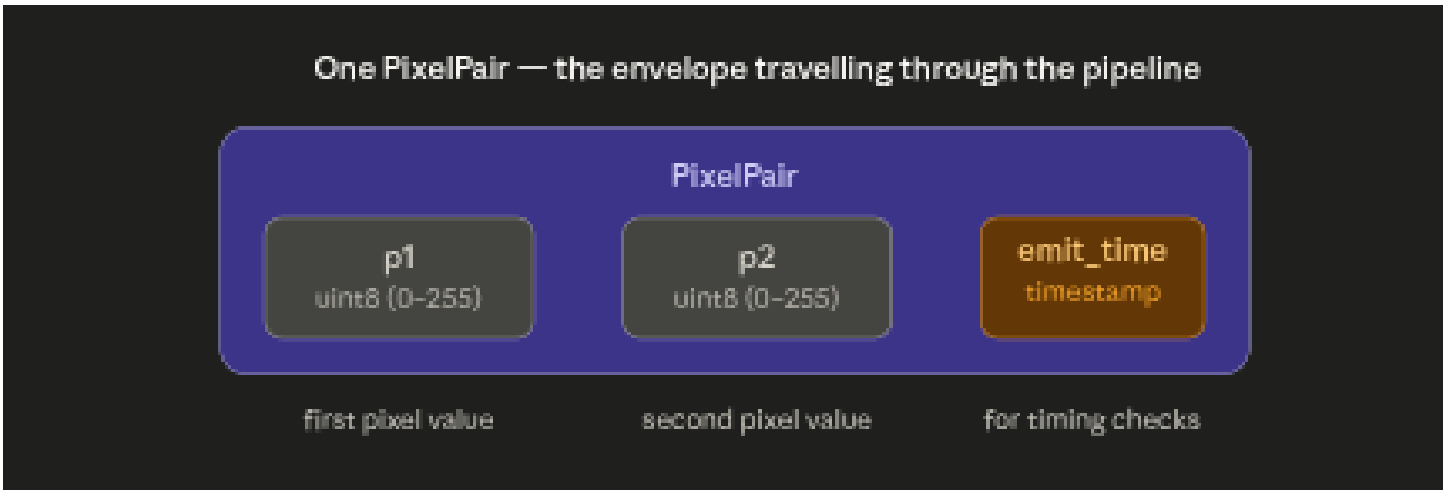


Image 5 in plain English: After filtering, the noisy jagged signal becomes a smooth curve. Now you draw an invisible horizontal line at height **TV** (threshold value — given by the user). For every pixel: if its filtered value is **above or touching** that line → output **1** (defect found!). If it's **below** that line → output **0** (all good). It's exactly like asking "is this pixel brighter than my cutoff level?" — a simple yes/no decision.

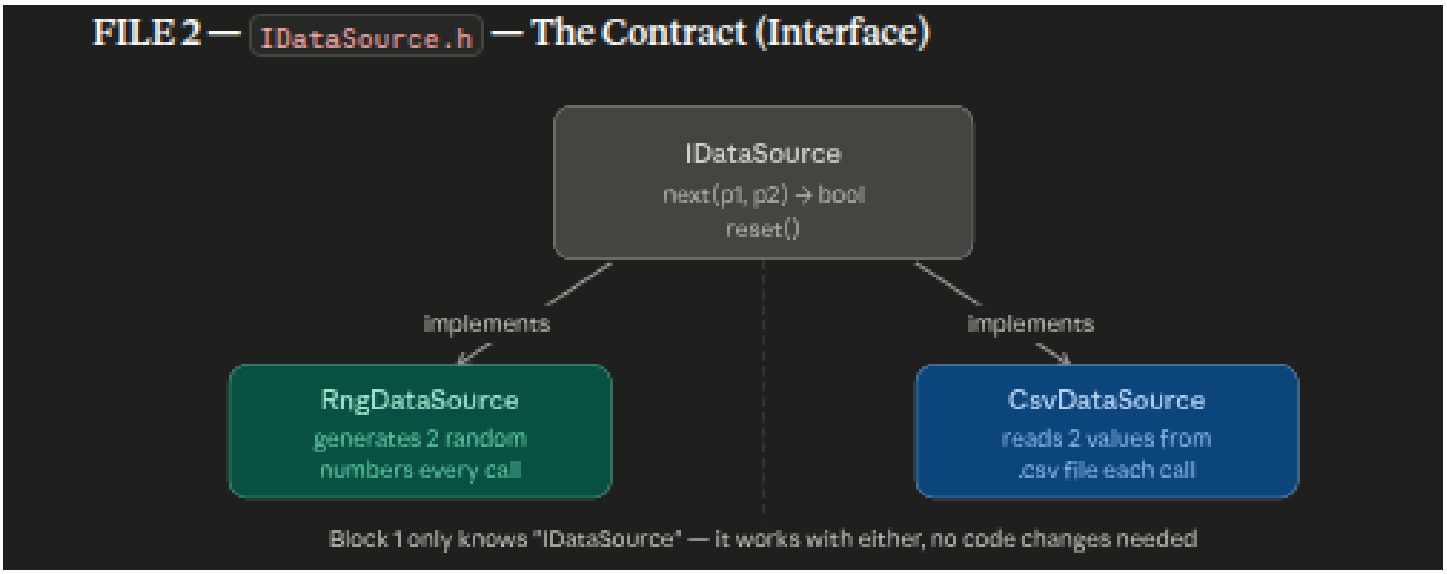
PROJECT STRUCTURE :

```
suneerth@suneerth-Lenovo-ideapad-330-15IKB:~/CynLr_Eval1$ tree
.
├── build
│   ├── CMakeCache.txt
│   ├── CMakeFiles
│   │   └── 3.22.1
│   │       ├── CMakeCXXCompiler.cmake
│   │       ├── CMakeDetermineCompilerABI_CXX.bin
│   │       ├── CMakeSystem.cmake
│   │       └── CompilerIdCXX
│   │           ├── a.out
│   │           ├── CMakeCXXCompilerId.cpp
│   │           └── tmp
│   ├── cmake.check_cache
│   ├── CMakeDirectoryInformation.cmake
│   ├── CMakeOutput.log
│   ├── CMakeTmp
│   ├── CynLr_Eval1.dir
│   │   ├── build.make
│   │   ├── cmake_clean.cmake
│   │   ├── compiler_depend.internal
│   │   ├── compiler_depend.make
│   │   ├── compiler_depend.ts
│   │   ├── DependInfo.cmake
│   │   ├── depend.make
│   │   ├── flags.make
│   │   ├── link.txt
│   │   ├── progress.make
│   │   └── src
│   │       ├── DataGenerationBlock.cpp.o
│   │       ├── DataGenerationBlock.cpp.o.d
│   │       ├── FilterThresholdBlock.cpp.o
│   │       ├── FilterThresholdBlock.cpp.o.d
│   │       ├── main.cpp.o
│   │       ├── main.cpp.o.d
│   │       ├── Pipeline.cpp.o
│   │       └── Pipeline.cpp.o.d
│   ├── Compiler.cpp.d
│   ├── Makefile2
│   ├── Makefile.cmake
│   ├── progress.marks
│   ├── run_tests.dir
│   ├── run_tests.dir
│   │   ├── build.make
│   │   ├── cmake_clean.cmake
│   │   ├── compiler_depend.internal
│   │   ├── compiler_depend.make
│   │   ├── compiler_depend.ts
│   │   ├── DependInfo.cmake
│   │   ├── depend.make
│   │   ├── depend.make
│   │   ├── flags.make
│   │   ├── link.txt
│   │   ├── progress.make
│   │   ├── src
│   │   │   ├── DataGenerationBlock.cpp.o
│   │   │   ├── DataGenerationBlock.cpp.o.d
│   │   │   ├── FilterThresholdBlock.cpp.o
│   │   │   ├── FilterThresholdBlock.cpp.o.d
│   │   │   ├── Pipeline.cpp.o
│   │   │   ├── Pipeline.cpp.o.d
│   │   │   └── tests
│   │   │       ├── tests.cpp.o
│   │   │       └── tests.cpp.o.d
│   │   ├── TargetDirectories.txt
│   │   ├── cmake_install.cmake
│   │   ├── DataGenerationBlock.cpp
│   │   ├── FilterThresholdBlock.cpp
│   │   ├── main.cpp
│   │   ├── Makefile
│   │   └── Pipeline.cpp
│   ├── tests
│   │   ├── tests.cpp.o
│   │   ├── CynLr_Eval1
│   │   ├── CynLr_Eval1
│   │   ├── Makefile
│   │   └── run_tests
│   ├── CMakeLists.txt
│   ├── data
│   │   └── sample.csv
│   ├── include
│   └── run_tests
├── CMakeLists.txt
└── data
    └── sample.csv
    ├── docs
    │   └── DesignOverview.docx
    └── src  BoundedQueue.h
        ├── CsvDataSource.h
        ├── DataGenerationBlock.h
        ├── IDataSource.h
        ├── Pipeline.h
        ├── PixelPair.h
        ├── PixelPair.h
        ├── RngDataSource.h
        ├──
        ├── FilterThresholdBlock.cpp
        └── src
            ├── main.cpp
            └── Pipeline.cpp
    └── {src,include,tests,data,docs,build}
        └── tests
            └── tests.cpp
```





PixelPair.h — What it is: Think of it as an **envelope**. Every time Block 1 produces 2 pixels, it stuffs them into this envelope and puts it on the queue for Block 2 to pick up. It carries 3 things: p1 (first pixel, 0–255), p2 (second pixel, 0–255), and emit_time (the exact clock time it was created — used to measure timing accuracy). Why uint8? Because pixel brightness is 0 (black) to 255 (white), and uint8 is exactly that range using only 1 byte of memory.



IDataSource.h — What it is: This is a **contract / promise**. It says: "whoever implements me MUST havFILE 3 — RngDataSource.h — The Random Number Generator

What it does: Uses C++'s std::mt19937 which is the Mersenne Twister — a high-quality random number generator (not rand() which is bad quality). It generates numbers uniformly between 0 and 255. Every call to next() gives 2 independent random numbers — simulating two ADC (Analog to Digital Converter) channels on a real camera. reset() re-seeds it with a new random seed for reproducibility in tests.

Key imports:

<random> — C++ random number library

std::mt19937 — the Mersenne Twister engine (fast, statistically good)

std::uniform_int_distribution<int>(0,255) — makes sure output is always 0–255e a next() function that gives 2 pixel values, and a reset() function." Block 1 doesn't care whether data comes from a CSV file or random numbers — it just calls next() and gets values. This is called the **Strategy Pattern** — you can swap the strategy (CSV vs random) without changing any other code. The = 0 after each function means "this is abstract — someone else must provide the actual code."

FILE 3 — RngDataSource.h — The Random Number Generator

What it does: Uses C++'s std::mt19937 which is the **Mersenne Twister** — a high-quality random number generator (not rand() which is bad quality). It generates numbers uniformly between 0 and 255. Every call to next() gives 2 independent random numbers — simulating two ADC (Analog to Digital Converter) channels on a real camera. reset() re-seeds it with a new random seed for reprodFILE 3 — RngDataSource.h — The Random Number Generator

What it does: Uses C++'s std::mt19937 which is the Mersenne Twister — a high-quality random number generator (not rand() which is bad quality). It generates numbers uniformly between 0 and 255. Every call to next() gives 2 independent random numbers — simulating two ADC (Analog to Digital Converter) channels on a real camera. reset() re-seeds it with a new random seed for reproducibility in tests.

Key imports:

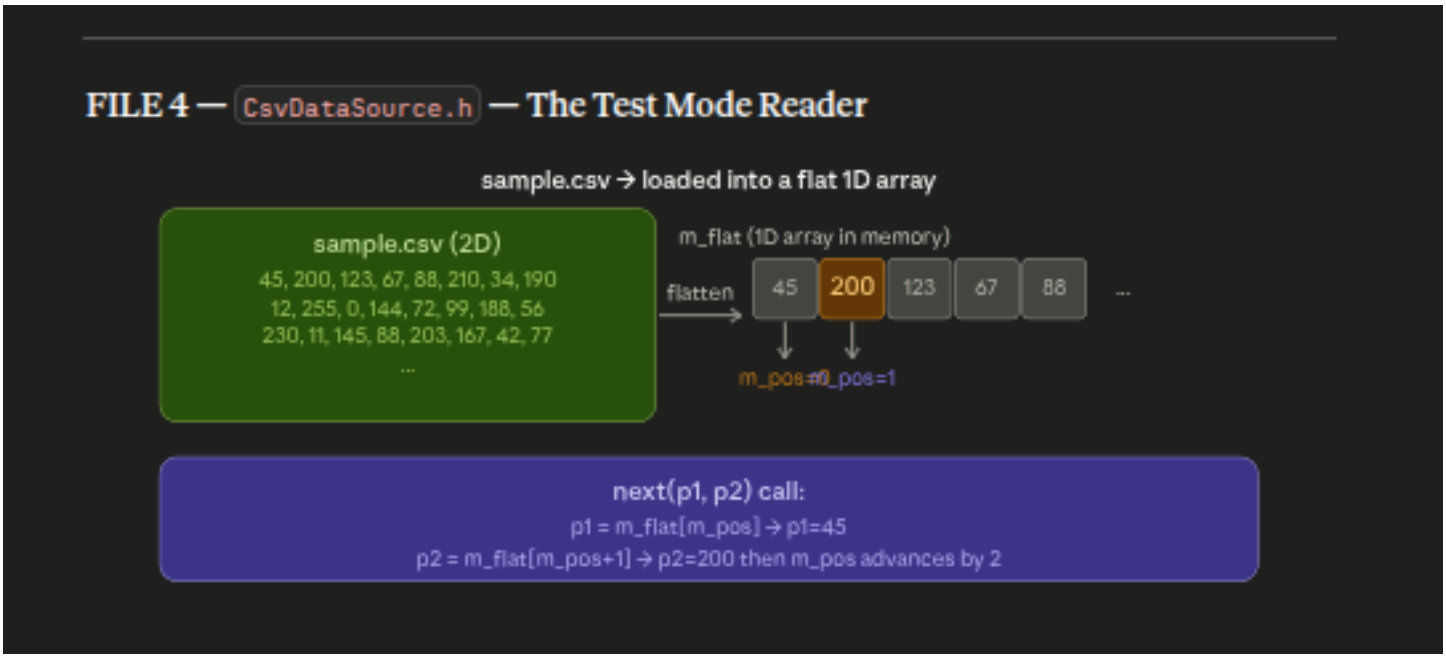
<random> — C++ random number library

std::mt19937 — the Mersenne Twister engine (fast, statistically good)

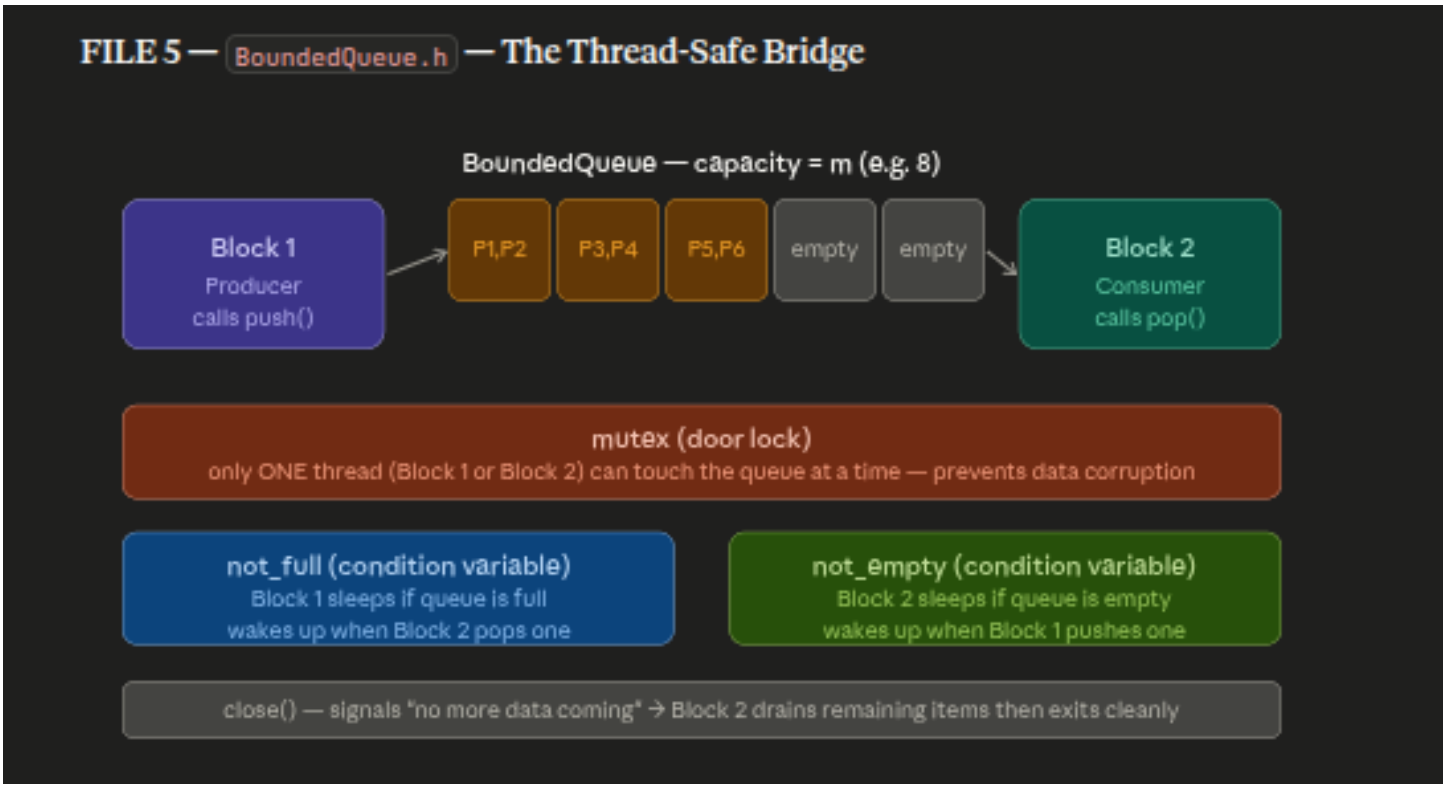
std::uniform_int_distribution<int>(0,255) — makes sure output is always 0–255ucibility in tests.

Key imports:

- <random> — C++ random number library
- std::mt19937 — the Mersenne Twister engine (fast, statistically good)
- std::uniform_int_distribution<int>(0,255) — makes sure output is always 0–255



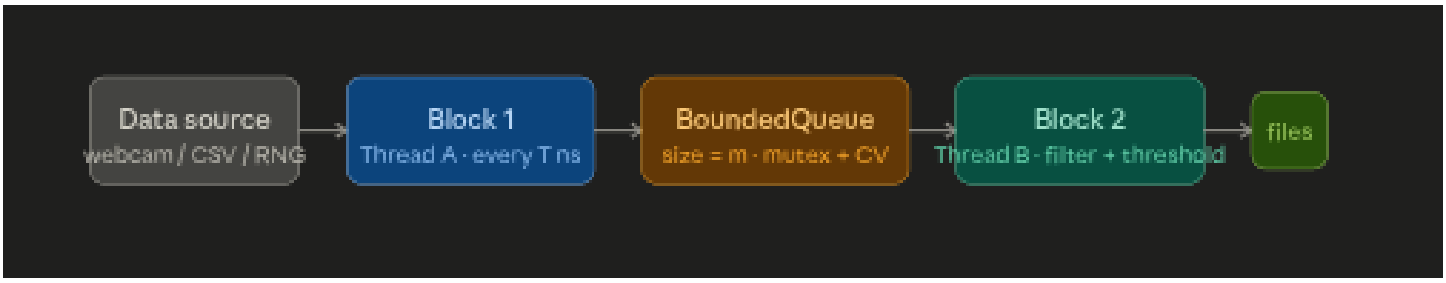
CsvDataSource.h — What it does: Reads the entire CSV file into a flat 1D array (m_flat) at startup. Then next() just moves a pointer (m_pos) forward by 2 each time — no file reading during live execution (fast!). When m_pos reaches the end, next() returns false — telling Block 1 the source is exhausted. Key validation: checks each value is 0–255, rejects if columns aren't even. reset() sets m_pos=0 to replay from beginning.



CynLr Line-Scan Camera Pipeline — Design Overview :

How the two blocks run together

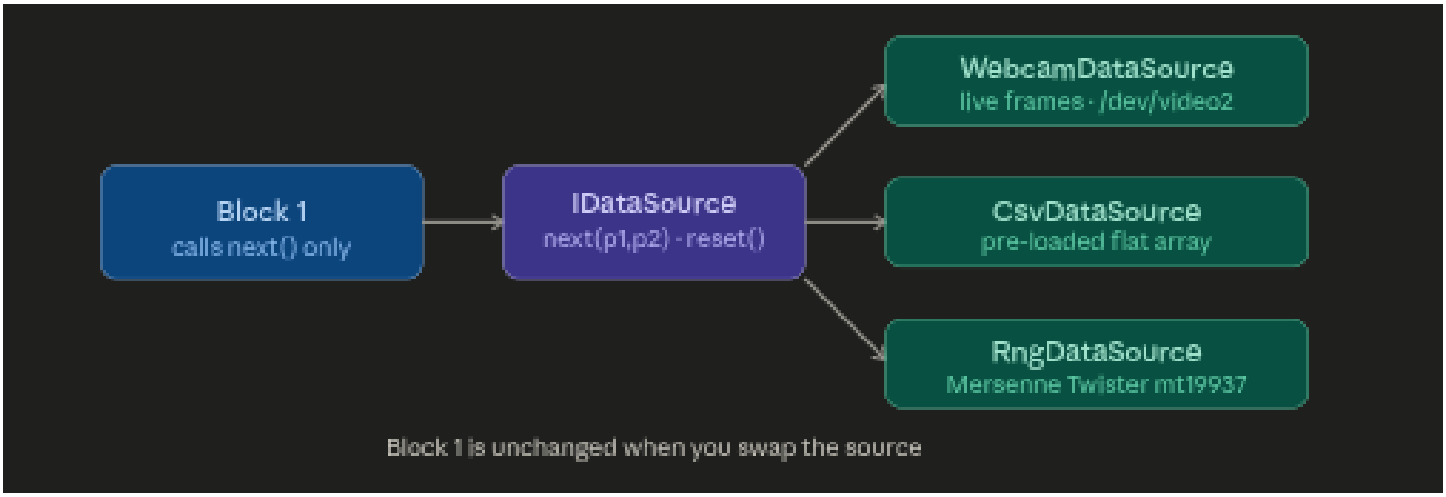
Block 1 (the camera) and Block 2 (the filter) are two workers running at the same time. Block 1 keeps producing pixel pairs every T nanoseconds. Block 2 keeps consuming and checking them. They never wait for each other — the queue in the middle handles the handoff.



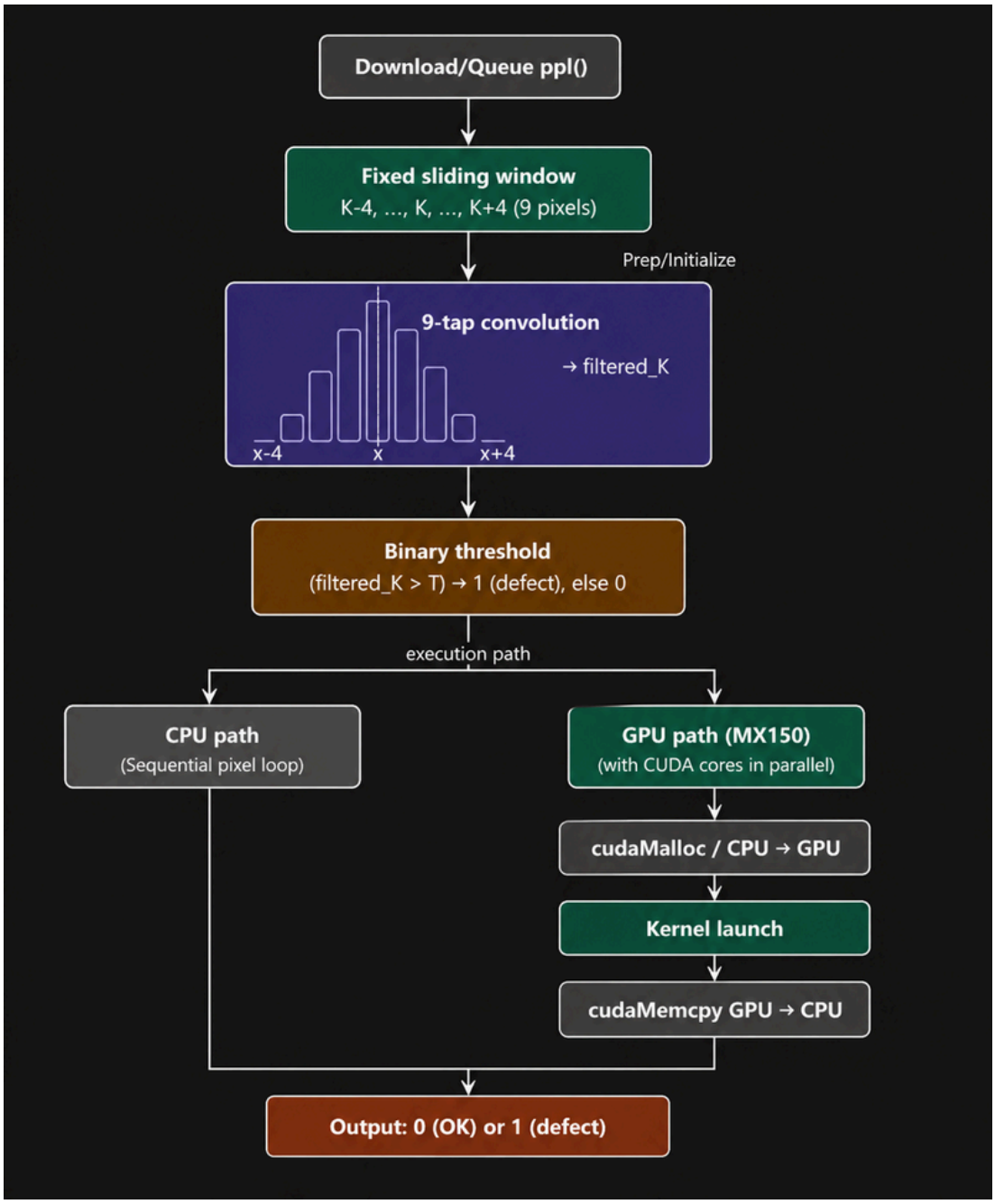
Block 1 — Data Generation (Thread A) :

Block 1 reads 2 pixels every T nanoseconds and pushes them into the queue. The key trick is using sleep_until(next_tick) instead of sleep(T) — a naive sleep drifts because it doesn't account for the time spent doing work. With sleep_until, the next tick is always exactly T nanoseconds after the previous one, regardless of how long reading the pixels took.

Block 1 knows nothing about where the pixels come from — it only calls next() on an IDataSource. This is the Strategy Pattern: swap webcam, CSV, or RNG without changing a single line of Block 1.



Block 2 — Filter & Threshold (Thread B) :



Block 2 runs in parallel with Block 1. It sits on the other end of the queue, constantly calling pop() to receive pixel pairs. For every pixel K it receives, it runs two operations in sequence: Gaussian convolution, then binary thresholding.

Step 1 — Gaussian convolution

Block 2 maintains a sliding window of 9 consecutive pixels (K-4 through K+4). Each pixel in the window gets multiplied by a weight from a fixed 9-tap kernel. The centre pixel K gets the largest weight (0.34376), the neighbours get progressively smaller weights, and the outermost pixels at ±4 positions get near-zero weight. The bar chart in the diagram above shows this bell-curve shape — it's exactly what Gaussian means.

The result, filtered_K, is a smoothed version of K. Random noise (a single bright or dark outlier) gets "averaged away" by its neighbours. A real defect — which spans multiple pixels — survives the filter because all 9 surrounding pixels are also elevated.

For edge pixels (the first 4 and last 4 in a row), there are no real neighbours to the left or right. The pipeline pads them by repeating the border pixel value — so the convolution can still run without special-casing.

Step 2 — Binary threshold

After filtering, a single comparison decides the output: if filtered_K ≥ TV, the pixel is flagged as a defect (output 1). If it's below TV, it's clean (output 0). TV is the threshold value you set with -tv on the command line — typically 128 for an 8-bit greyscale signal.

GPU vs CPU execution path :

When CUDA is available (the MX150 in this project), Block 2 doesn't process pixels one at a time. Instead it accumulates a batch of 1024 pixels, ships them to the GPU with cudaMemcpy, launches a kernel across 4 blocks × 256 threads (so 1024 threads run simultaneously), each thread independently runs the full 9-tap convolution and threshold comparison on its one pixel, then the results come back to CPU memory. On the MX150 this is roughly 5–10× faster than the sequential CPU loop for the same batch.

Shutdown and results

When the user hits Ctrl+C, Block 1 closes the queue. Block 2's next pop() returns a null signal, it exits its loop, flushes any remaining pixels (tail flush on CPU, final batch on GPU), and saves everything to data/run_<timestamp>/ — the webcam frame PNG, results.csv with every pixel's raw value, filtered value, and defect flag, defects_only.csv, and the human-readable inference_report.txt.

Design patterns at work in Block 2

The OutputCallback lambda is the Observer pattern — Block 2 emits results without knowing or caring whether they're being written to CSV, printed to terminal, or discarded. The RAII destructors ensure GPU memory and any open file handles are always released cleanly, even if the thread exits on an exception. The CPU/GPU branching is controlled at startup by whether CUDA was found at build time, so the same source compiles cleanly on machines with no GPU at all.